Kotlin Closures

50 Comprehensive Examples with Different Patterns

Master closures in Kotlin: variable capture, function factories, currying, memoization, callbacks, DSLs, state machines, observers, middleware, caching, rate limiting, and much more. Understand how closures enable powerful functional programming patterns.

BASIC CLOSURES (1-15)

Example 1: Simple Variable Capture

```
fun main() { var count = 0 val increment = { count++ // Captures and modifies 'count' println("Count: $count") } increment() // Count: 1 increment() // Count: 2 increment() // Count: 3 println("Final count: $count") // Final count: 3 }
```

■ *Closure captures and modifies external variable - basic closure behavior*

Example 2: Counter with Closure

```
fun createCounter(): () -> Int { var count = 0 return { count++ count } } fun main() { val counter1 = createCounter() val counter2 = createCounter() println(counter1()) // 1 println(counter1()) // 2 println(counter1()) // 3 println(counter2()) // 1 (separate state) println(counter2()) // 2 }
```

■ *Each closure maintains its own captured state - independent counters*

Example 3: Closure Returning Function

```
fun multiplier(factor: Int): (Int) -> Int { return { number -> number * factor // Captures 'factor' } } fun main() { val double = multiplier(2) val triple = multiplier(3) val times10 = multiplier(10) println(double(5)) // 10 println(triple(5)) // 15 println(times10(5)) // 50 println(double(8)) // 16 }
```

■ *Closure captures parameter - creates specialized functions*

Example 4: Closure with Multiple Captures

```
fun main() { var x = 10 var y = 20 var operation = "add" val calculate = { when (operation) { "add" -> x + y "subtract" -> x - y "multiply" -> x * y else -> 0 } } println(calculate()) // 30 operation = "multiply" println(calculate()) // 200 x = 5 y = 3 println(calculate()) // 15 }
```

■ *Closure captures multiple variables - reflects all changes*

Example 5: List Filter with Closure

```
fun createFilter(threshold: Int): (Int) -> Boolean { return { number -> number > threshold } } fun main() { val numbers = listOf(1, 5, 10, 15, 20, 25, 30) val above10 = createFilter(10) val above20 = createFilter(20) println(numbers.filter(above10)) // [15, 20, 25, 30] println(numbers.filter(above20)) // [25, 30] // Inline closure val threshold = 15 val filtered = numbers.filter { it > threshold } println(filtered) // [20, 25, 30] }
```

■ *Closures as predicates - captures threshold for filtering*

Example 6: Accumulator Pattern

```
fun createAccumulator(initial: Int): (Int) -> Int { var sum = initial return { value -> sum += value sum } } fun main() { val acc1 = createAccumulator(0) val acc2 = createAccumulator(100) println(acc1(5)) // 5 println(acc1(10)) // 15 println(acc1(3)) // 18 println(acc2(5)) // 105 println(acc2(10)) // 115 }
```

■ *Accumulator maintains running sum - each instance independent*

Example 7: Event Handler with Context

```
class Button(val label: String) { private var clickHandler: (() -> Unit)? = null fun setOnClick(handler: () -> Unit) { clickHandler = handler } fun click() { println("Button '$label' clicked") clickHandler?.invoke() } } fun main() { var clickCount = 0 val button = Button("Submit") button.setOnClick { clickCount++ // Closure captures clickCount println("Clicked $clickCount times") } button.click() // Clicked 1 times button.click() // Clicked 2 times button.click() // Clicked 3 times }
```

■ *Closure in event handling - maintains state across invocations*

Example 8: Private Variable Encapsulation

```
fun createPerson(name: String) = object { private var _name = name private var _age = 0 val getName = { _name } val setName = { newName: String -> _name = newName } val getAge = { _age } val setAge = { newAge: Int -> if (newAge >= 0) _age = newAge } val introduce = { "Hi, I'm ${_name}, ${_age} years old" } } fun main() { val person = createPerson("Alice") println(person.getName()) // Alice person.setAge(25) println(person.introduce()) // Hi, I'm Alice, 25 years old person.setName("Bob") println(person.introduce()) // Hi, I'm Bob, 25 years old }
```

■ *Closures for encapsulation - creates private state with public interface*

Example 9: Memoization with Closure

```
fun <T, R> memoize(fn: (T) -> R): (T) -> R { val cache = mutableMapOf<T, R>() return { input -> cache.getOrPut(input) { println("Computing for $input") fn(input) } } } fun expensiveOperation(n: Int): Int { Thread.sleep(1000) // Simulate expensive computation return n * n } fun main() { val memoized = memoize(::expensiveOperation) println(memoized(5)) // Computing for 5, then 25 println(memoized(5)) // 25 (from cache, instant) println(memoized(3)) // Computing for 3, then 9 println(memoized(5)) // 25 (from cache) }
```

■ *Closure maintains cache - performance optimization pattern*

Example 10: Partial Application

```
fun add(a: Int, b: Int, c: Int) = a + b + c fun partial2(fn: (Int, Int, Int) -> Int, a: Int): (Int, Int) -> Int { return { b, c -> fn(a, b, c) } } fun partial3(fn: (Int, Int, Int) -> Int, a: Int, b: Int): (Int) -> Int { return { c -> fn(a, b, c) } } fun main() { val add5 = partial2(::add, 5) println(add5(10, 15)) // 30 val add5And10 = partial3(::add, 5, 10) println(add5And10(20)) // 35 println(add5And10(100)) // 115 }
```

■ *Closure for partial application - creates specialized functions*

Example 11: Lazy Initialization

```
class ExpensiveResource { init { println("Resource created!") Thread.sleep(500) } fun use() = println("Using resource") } fun lazyInit(initializer: () -> ExpensiveResource): () -> ExpensiveResource { var resource: ExpensiveResource? = null return { if (resource == null) { println("Initializing...") resource = initializer() } resource!! } } fun main() { val getResource = lazyInit { ExpensiveResource() } println("Before first use") getResource().use() // Initializing... Resource created! Using resource println("Second use") getResource().use() // Using resource (no re-initialization) }
```

■ *Closure delays expensive initialization - lazy loading pattern*

Example 12: Closure in Loop (Common Pitfall)

```
fun main() { // WRONG WAY - Common mistake val functions = mutableListOf<() -> Unit>() for (i in 1..3) { functions.add { println("Wrong: $i") } } functions.forEach { it() } // Wrong: 3, Wrong: 3, Wrong: 3 (all capture same 'i') // CORRECT WAY - Capture current value val correctFunctions = mutableListOf<() -> Unit>() for (i in 1..3) { val captured = i // Create new variable correctFunctions.add { println("Correct: $captured") } } correctFunctions.forEach { it() } // Correct: 1, Correct: 2, Correct: 3 }
```

■ *Loop closure pitfall - demonstrates variable capture timing*

Example 13: Timer with Closure State

```
import kotlin.concurrent.timer fun createTimer(name: String) { var seconds = 0 timer(period = 1000, initialDelay = 0) { seconds++ println("$name: $seconds seconds") if (seconds >= 3) { println("$name stopped") cancel() } } } fun main() { var globalCount = 0 timer(period = 500, initialDelay = 0) { globalCount++ println("Tick $globalCount") if (globalCount >= 5) { cancel() } } Thread.sleep(3000) }
```

■ *Closure in async context - maintains state across timer events*

Example 14: Callback with Captured Context

```
class AsyncTask { fun execute( onStart: () -> Unit, onProgress: (Int) -> Unit, onComplete: (String) -> Unit ) { onStart() for (i in 1..5) { Thread.sleep(200) onProgress(i * 20) } onComplete("Task finished!") } } fun main() { var taskName = "Data Download" var startTime = 0L AsyncTask().execute( onStart = { startTime = System.currentTimeMillis() println("$taskName started") }, onProgress = { progress -> println("$taskName: $progress%") }, onComplete = { result -> val duration = System.currentTimeMillis() - startTime println("$taskName completed: $result") println("Duration: ${duration}ms") } ) }
```

■ *Multiple closures share captured variables - callback pattern*

Example 15: Debounce Function

```
import kotlinx.coroutines.* fun <T> debounce( waitMs: Long, scope: CoroutineScope, action: (T) -> Unit ): (T) -> Unit { var debounceJob: Job? = null return { param: T -> debounceJob?.cancel() debounceJob = scope.launch { delay(waitMs) action(param) } } } fun main() = runBlocking { val search = debounce<String>(500, this) { query -> println("Searching for: $query") } // Rapid calls - only last one executes search("a") delay(100) search("ab") delay(100) search("abc") delay(1000) // Wait for debounce }
```

■ *Closure for debouncing - captures job reference for cancellation*

INTERMEDIATE PATTERNS (16-30)

Example 16: Closure Composition

```
fun <A, B, C> compose( f: (B) -> C, g: (A) -> B ): (A) -> C { return { a -> f(g(a)) } } fun main() { val double: (Int) -> Int = { it * 2 } val addTen: (Int) -> Int = { it + 10 } val toString: (Int) -> String = { "Result: $it" } val composed = compose(toString, compose(double, addTen)) println(composed(5)) // Result: 30 // (5 + 10) * 2 = 30 val pipeline = compose( compose(toString, double), addTen ) println(pipeline(5)) // Result: 30 }
```

■ *Function composition with closures - builds complex operations*

Example 17: Currying

```
fun curry3(fn: (Int, Int, Int) -> Int): (Int) -> (Int) -> (Int) -> Int { return { a -> { b -> { c -> fn(a, b, c) } } } } fun sum(a: Int, b: Int, c: Int) = a + b + c fun main() { val curriedSum = curry3(::sum) // Partial application step by step val add5 = curriedSum(5) val add5And10 = add5(10) println(add5And10(20)) // 35 println(add5And10(100)) // 115 // All at once println(curriedSum(1)(2)(3)) // 6 }
```

■ *Currying transforms multi-param function to chain of closures*

Example 18: Builder Pattern with DSL

```
class HtmlBuilder { private val elements = mutableListOf<String>() fun tag(name: String, content: String) { elements.add("<$name>$content</$name>") } fun tag(name: String, builder: HtmlBuilder.() -> Unit) { val nested = HtmlBuilder() nested.builder() elements.add("<$name>${nested.build()}</$name>") } fun build() = elements.joinToString("") } fun html(builder: HtmlBuilder.() -> Unit): String { val htmlBuilder = HtmlBuilder() htmlBuilder.builder() return htmlBuilder.build() } fun main() { val page = html { tag("html") { tag("head") { tag("title", "My Page") } tag("body") { tag("h1", "Welcome") tag("p", "Hello World!") } } } println(page) }
```

■ *Closure with receiver - creates type-safe DSL builders*

Example 19: Recursive Closure

```
fun factorial(): (Int) -> Int { lateinit var fac: (Int) -> Int fac = { n -> if (n <= 1) 1 else n * fac(n - 1) // Closure captures itself } return fac } fun fibonacci(): (Int) -> Int { val cache = mutableMapOf<Int, Int>() lateinit var fib: (Int) -> Int fib = { n -> cache.getOrPut(n) { when (n) { 0 -> 0 1 -> 1 else -> fib(n - 1) + fib(n - 2) } } } return fib } fun main() { val fact = factorial() println(fact(5)) // 120 println(fact(10)) // 3628800 val fib = fibonacci() println(fib(10)) // 55 println(fib(20)) // 6765 }
```

■ *Self-referencing closure - enables recursion with captured cache*

Example 20: State Machine with Closures

```
enum class TrafficLight { RED, YELLOW, GREEN } fun createTrafficLightController() = object { private var currentState = TrafficLight.RED private var changeCount = 0 val next = { currentState = when (currentState) { TrafficLight.RED -> TrafficLight.GREEN TrafficLight.GREEN -> TrafficLight.YELLOW TrafficLight.YELLOW -> TrafficLight.RED } changeCount++ println("Changed to $currentState (change #$changeCount)") currentState } val current = { currentState } val getChangeCount = { changeCount } val reset = { currentState = TrafficLight.RED changeCount = 0 } } fun main() { val light = createTrafficLightController() repeat(5) { light.next() } println("Total changes: ${light.getChangeCount()}") }
```

■ *State machine using closures - encapsulates state transitions*

Example 21: Observer Pattern with Closures

```
class Observable<T>(initial: T) { private var value: T = initial private val observers = mutableListOf<(T) -> Unit>() fun subscribe(observer: (T) -> Unit): () -> Unit { observers.add(observer) observer(value) // Immediately notify with current value // Return unsubscribe function (closure) return { observers.remove(observer) } } fun set(newValue: T) { value = newValue observers.forEach { it(value) } } } fun main() { val temperature = Observable(20.0) var displayTemp = "" var alerts = 0 val unsubDisplay = temperature.subscribe { temp -> displayTemp = "Temperature: ${temp}°C" println(displayTemp) } val unsubAlert = temperature.subscribe { temp -> if (temp > 30) { alerts++ println("■■ HIGH TEMPERATURE ALERT!") } } temperature.set(25.0) temperature.set(35.0) // Triggers alert unsubAlert() // Unsubscribe from alerts temperature.set(40.0) // No alert }
```

■ *Closures maintain observer list and return unsubscribe closures*

Example 22: Throttle Function

```
import kotlinx.coroutines.* fun <T> throttle( intervalMs: Long, scope: CoroutineScope, action: (T) -> Unit ): (T) -> Unit { var lastRun = 0L var pendingJob: Job? = null return { param: T -> val now = System.currentTimeMillis() val timeSinceLastRun = now - lastRun if (timeSinceLastRun >= intervalMs) { lastRun = now action(param) } else { pendingJob?.cancel() pendingJob = scope.launch { delay(intervalMs - timeSinceLastRun) lastRun = System.currentTimeMillis() action(param) } } } } fun main() = runBlocking { val logClick = throttle<String>(1000, this) { button -> println("Click: $button at ${System.currentTimeMillis()}") } repeat(10) { i -> logClick("Button$i") delay(300) } delay(2000) }
```

■ *Throttling with closure state - limits execution rate*

Example 23: Retry Logic with Closure

```
suspend fun <T> retry( times: Int, delayMs: Long = 1000, block: suspend () -> T ): Result<T> { var lastException: Exception? = null var attempt = 0 repeat(times) { attempt++ try { return Result.success(block()) } catch (e: Exception) { lastException = e println("Attempt $attempt failed: ${e.message}") if (attempt < times) { delay(delayMs) } } } return Result.failure(lastException ?: Exception("Unknown error")) } suspend fun unreliableOperation(): String { if (Math.random() < 0.7) throw Exception("Random failure") return "Success!" } fun main() = runBlocking { val result = retry(5, 500) { unreliableOperation() } result.fold( onSuccess = { println("Result: $it") }, onFailure = { println("Failed after retries: ${it.message}") } ) }
```

■ *Closure captures retry state - resilient operation execution*

Example 24: Fluent API with Closures

```
class QueryBuilder { private var table = "" private val columns = mutableListOf<String>() private var whereClause = "" private var orderBy = "" fun from(tableName: String) = apply { table = tableName } fun select(vararg cols: String) = apply { columns.addAll(cols) } fun where(condition: () -> String) = apply { whereClause = condition() } fun orderBy(column: String, desc: Boolean = false) = apply { orderBy = "$column ${if (desc) "DESC" else "ASC"}" } fun build(): String { val cols = if (columns.isEmpty()) "*" else columns.joinToString() var sql = "SELECT $cols FROM $table" if (whereClause.isNotEmpty()) sql += " WHERE $whereClause" if (orderBy.isNotEmpty()) sql += " ORDER BY $orderBy" return sql } } fun main() { var minAge = 18 var maxAge = 65 val query = QueryBuilder() .select("name", "email", "age") .from("users") .where { "age BETWEEN $minAge AND $maxAge" } .orderBy("age", desc = true) .build() println(query) // SELECT name, email, age FROM users WHERE age BETWEEN 18 AND 65 ORDER BY age DESC }
```

■ *Closures in fluent interfaces - dynamic query building*

Example 25: Pipeline Processing

```
fun <T> pipeline(vararg steps: (T) -> T): (T) -> T { return { input -> steps.fold(input) { acc, step -> step(acc) } } } fun main() { val processText = pipeline<String>( { it.trim() }, { it.lowercase() }, { it.replace("bad", "***") }, { it.split(" ").joinToString(" ") { word -> word.replaceFirstChar { it.uppercase() } } } ) val input = " Hello BAD World " val result = processText(input) println(result) // Hello *** World val processNumbers = pipeline<Int>( { it * 2 }, { it + 10 }, { it * it } ) println(processNumbers(5)) // ((5*2)+10)^2 = 400 }
```

■ *Function pipeline with closures - composable data transformations*

Example 26: Validation Chain

```
class Validator<T> { private val rules = mutableListOf<(T) -> String?>() fun addRule(rule: (T) -> String?) = apply { rules.add(rule) } fun validate(value: T): List<String> { return rules.mapNotNull { it(value) } } } fun createEmailValidator() = Validator<String>() .addRule { if (it.isBlank()) "Email is required" else null } .addRule { if (!it.contains("@")) "Invalid email format" else null } .addRule { if (it.length < 5) "Email too short" else null } fun createPasswordValidator(minLength: Int) = Validator<String>() .addRule { if (it.length < minLength) "Password must be $minLength+ chars" else null } .addRule { if (!it.any { c -> c.isDigit() }) "Must contain number" else null } .addRule { if (!it.any { c -> c.isUpperCase() }) "Must contain uppercase" else null } fun main() { val emailValidator = createEmailValidator() val passwordValidator = createPasswordValidator(8) println(emailValidator.validate("")) // [Email is required] println(emailValidator.validate("test")) // [Invalid email format] println(emailValidator.validate("a@b.com")) // [] println(passwordValidator.validate("weak")) // [Password must be 8+ chars, Must contain number, Must contain uppercase] }
```

■ *Validation closures - flexible, composable validation rules*

Example 27: Middleware Pattern

```
typealias Middleware<T> = (T, () -> T) -> T fun <T> createMiddlewareChain(vararg middlewares: Middleware<T>): (T) -> T { return { initial -> middlewares.foldRight({ initial }) { middleware, next -> { middleware(it, next) } }(initial) } } fun main() { val loggingMiddleware: Middleware<String> = { value, next -> println("Before: $value") val result = next() println("After: $result") result } val uppercaseMiddleware: Middleware<String> = { value, next -> next().uppercase() } val trimMiddleware: Middleware<String> = { value, next -> next().trim() } val process = createMiddlewareChain( loggingMiddleware, uppercaseMiddleware, trimMiddleware ) val result = process(" hello world ") println("Final: $result") }
```

■ *Middleware chain with closures - modular request processing*

Example 28: Caching Decorator

```
fun <K, V> cached(fn: (K) -> V): (K) -> V { val cache = mutableMapOf<K, V>() var hits = 0 var misses = 0 val cachedFn = { key: K -> cache.getOrPut(key) { misses++ fn(key) }.also { hits++ } } val printStats = { println("Cache stats - Hits: $hits, Misses: $misses, Size: ${cache.size}") } // Attach stats as property (hacky but demonstrates closure) cachedFn.also { (it as? Any)?.let { printStats } } return cachedFn } fun expensiveCalculation(n: Int): Int { println("Computing for $n...") Thread.sleep(500) return n * n } fun main() { val compute = cached(::expensiveCalculation) println(compute(5)) // Computing for 5... 25 println(compute(5)) // 25 (cached) println(compute(3)) // Computing for 3... 9 println(compute(5)) // 25 (cached) }
```

■ *Caching with closure - transparent performance optimization*

Example 29: Rate Limiter

```
import java.util.concurrent.ConcurrentLinkedQueue import kotlinx.coroutines.* fun <T> rateLimited( maxCalls: Int, windowMs: Long, scope: CoroutineScope, action: suspend (T) -> Unit ): suspend (T) -> Unit { val callTimes = ConcurrentLinkedQueue<Long>() return { param: T -> val now = System.currentTimeMillis() // Remove old calls outside window while (callTimes.peek()?.let { now - it > windowMs } == true) { callTimes.poll() } if (callTimes.size < maxCalls) { callTimes.offer(now) action(param) } else { val oldestCall = callTimes.peek() ?: now val waitTime = windowMs - (now - oldestCall) println("Rate limit hit, waiting ${waitTime}ms") delay(waitTime) callTimes.poll() callTimes.offer(System.currentTimeMillis()) action(param) } } } fun main() = runBlocking { val apiCall = rateLimited<String>(3, 2000, this) { request -> println("API Call: $request at ${System.currentTimeMillis()}") } repeat(5) { i -> apiCall("Request $i") } }
```

■ *Rate limiting with closure - protects against excessive calls*

Example 30: Transaction Wrapper

```
class Database { private var inTransaction = false private val operations = mutableListOf<String>() fun <T> transaction(block: Database.() -> T): T { if (inTransaction) { return block() // Nested transaction } inTransaction = true operations.clear() return try { val result = block() commit() result } catch (e: Exception) { rollback() throw e } finally { inTransaction = false } } fun insert(data: String) { operations.add("INSERT: $data") } fun update(data: String) { operations.add("UPDATE: $data") } private fun commit() { println("Committing: ${operations.joinToString()}") } private fun rollback() { println("Rolling back: ${operations.joinToString()}") operations.clear() } } fun main() { val db = Database() db.transaction { insert("User 1") insert("User 2") update("User 1 modified") } // Auto-commits try { db.transaction { insert("User 3") throw Exception("Error!") } } catch (e: Exception) { println("Transaction failed: ${e.message}") } // Auto-rollback }
```

■ *Closure for transaction scope - automatic commit/rollback*